

64-56 Projet de développement sur mandat

Tests

2023 - 2024

Groupe « Les Dat'Accros-bates »

Christophe Alexakis

Florian Naef

Pamela Verdon

Faton Zumeri

HEG – Haute Ecole de Gestion

Genève

Table des changements

Version	Date	Auteur / Autrice	Sous-titres / Chapitres	Description
V0	03.05.2024	Pamela		Version originale
V1	25.05.2024	Pamela	Réussite des Tests	Ajout de la section avec une capture d'écran prouvant la bonne exécution des tests
V1	29.05.2024	Pamela	Vue d'ensemble des tests → Problèmes rencontrés	Explications ajoutées concernant les tests reliés à Firebase, après une nouvelle tentative

Table des matières

Introduction	4
Vue d'ensemble des tests	5
Dépendances utilisées.....	5
Tests unitaires et d'intégration	5
Problèmes rencontrés.....	6
Description des composants et fonctions testés	7
BoutonOnglets	7
Theme	7
generateRandomPassword (fonction)	7
ConcoursListe	7
GymnasteListe	7
SocieteListe	7
UtilisateurListe	7
ResultatListe	7
Détails des tests	8
BoutonOnglets	8
Test pour vérifier si le composant s'affiche correctement avec les onglets donnés	8
Test pour vérifier si le bouton correspondant à l'emplacement actuel est activé	8
Test pour vérifier si le clic sur un bouton de l'onglet navigue vers la route correcte.....	8
Theme	9
Test pour vérifier l'affichage initial du composant.....	9
Test pour vérifier le basculement vers le mode sombre.....	9
Test pour vérifier le basculement vers le mode clair	9
generateRandomPassword	10
Test de la génération d'un mot de passe aléatoire de la longueur spécifiée.....	10
ConcoursListe	11
Test d'affichage correct de la liste des concours	11
Tests du tri des concours par date (ascendante et descendante)	11
Test de la sélection d'un concours	11
Test de l'état de la carte cliquée	11
GymnasteListe	12
Test d'affichage correct de la liste des gymnastes.....	12
Test de l'affichage de la liste des gymnastes lorsque display est false	12
Test du filtrage des gymnastes en fonction de la recherche	12
Test du tri des gymnastes en fonction de l'option de tri sélectionnée (nom)	12
Test du tri des gymnastes en fonction de l'option de tri sélectionnée (catégorie)	13

Test de la sélection d'un gymnaste	13
Test de l'ajout de la classe "cliquee" à la carte lorsqu'elle est cliquée	13
SocieteListe	14
Test d'affichage correct de la liste des sociétés	14
Test du filtre des sociétés en fonction de la recherche	14
Test du tri des sociétés en fonction de l'option de tri sélectionnée	14
Test de la sélection d'une société	14
Test de l'état de la carte cliquée	15
UtilisateurListe	16
Test d'affichage correct de la liste des utilisateurs	16
Test du filtre des utilisateurs en fonction de la recherche.....	16
Test du tri des utilisateurs en fonction de l'option de tri sélectionnée (nom).....	16
Test du tri des utilisateurs en fonction de l'option de tri sélectionnée (e-mail).....	16
Test de la sélection d'un utilisateur	17
Test de l'état de la carte cliquée	17
ResultatListe	18
Test d'affichage correct des résultats pour chaque engin	18
Test de tri des résultats par clé d'engin dans l'ordre alphabétique	18
Test de calcul correct du résultat total	18
Test d'affichage correct du classement et des récompenses	18
Test d'affichage correct des résultats d'équipe si fournis.....	19
Réussite des Tests	20
Conclusion	21

Introduction

Ce document a pour objectif de fournir une documentation détaillée des tests effectués sur différents composants et fonctionnalités de notre application web. Dans le cadre du développement de cette application, il est essentiel de garantir la qualité et la fiabilité de chaque composant et fonctionnalité, afin de fournir une expérience utilisateur optimale.

L'objectif principal de ces tests est de s'assurer que chaque composant fonctionne comme prévu, en conformité avec les spécifications définies et les attentes des utilisateurs finaux et du mandant. Ils visent également à identifier et à corriger tout dysfonctionnement ou comportement inattendu qui pourrait compromettre la performance de l'application.

Nos tests se trouvent dans le dossier "src/tests".

Vue d'ensemble des tests

Dépendances utilisées

Jest est le principal framework de test que nous utilisons pour écrire et exécuter nos tests JavaScript. Il offre un ensemble d'outils pour effectuer des assertions, simuler des comportements, et évaluer le bon fonctionnement de notre code.

En complément, nous utilisons les bibliothèques `@testing-library/jest-dom` et `@testing-library/react`. `@testing-library/jest-dom` étend les fonctionnalités de Jest en fournissant des outils supplémentaires pour effectuer des assertions sur le DOM de manière claire et concise. Quant à `@testing-library/react`, elle simplifie le processus de test des composants React en offrant une approche centrée sur l'utilisateur, ce qui nous permet de tester nos composants dans des scénarios d'utilisation réelle.

Tests unitaires et d'intégration

Les tests unitaires se concentrent sur la vérification du bon fonctionnement des unités individuelles de code, telles que des fonctions ou des composants, dans un environnement isolé. Ils visent à garantir que chaque unité fonctionne correctement selon ses spécifications, sans dépendre d'autres parties du système.

D'autre part, les tests d'intégration évaluent la manière dont plusieurs unités de code interagissent ensemble et s'intègrent pour former des fonctionnalités complètes. Ils examinent la communication et la coordination entre différents composants ou modules de l'application, afin de garantir leur bon fonctionnement en tant qu'ensemble.

La portée des tests couvre les fonctionnalités suivantes :

Tests unitaires :

BoutonOnglets : Test de l'affichage correct des boutons d'onglets et de leur fonctionnalité de navigation.

Theme : Validation du basculement entre les modes sombre et clair de l'application via le composant de thème.

En outre, la fonction *generateRandomPassword* est testée pour garantir la génération correcte de mots de passe aléatoires d'une longueur spécifiée. Ce test unitaire contribue à assurer la sécurité et la fiabilité des mécanismes d'authentification de l'application.

Tests d'intégration :

ConcoursListe : Vérification de l'affichage et du fonctionnement corrects de la liste des concours, y compris le tri et la sélection.

GymnasteListe : Test de l'affichage et du fonctionnement des listes de gymnastes, avec des fonctionnalités telles que la recherche, le tri et la sélection.

SocieteListe : Validation de l'affichage et du fonctionnement des listes de sociétés, incluant la recherche, le tri et la sélection.

UtilisateurListe : Vérification de l'affichage et du fonctionnement des listes d'utilisateurs, avec des fonctionnalités similaires à celles de la liste de sociétés.

ResultatListe : Test complet des fonctionnalités d'affichage et de gestion des résultats des gymnastes pour un concours spécifié.

Problèmes rencontrés

Malgré nos efforts pour tester les différentes composantes et fonctionnalités essentielles de l'application, nous avons rencontré des difficultés significatives lors de la mise en œuvre des tests pour les méthodes backend, en particulier celles impliquant l'interaction avec la base de données Firestore.

Les tests unitaires visant à valider les opérations telles que l'ajout, la modification et la récupération de données dans Firestore ont échoué. Nous avons constaté que les méthodes spécifiques à Firestore, telles que *getDocs*, ne fonctionnaient pas correctement dans l'environnement de test. Malgré nos tentatives pour résoudre ces problèmes, la simulation d'une base de données locale pour contourner ces difficultés s'est révélée complexe.

Bien que Firebase propose une solution sous le nom de "Firebase Local Emulator Suite", permettant de simuler localement une base de données similaire à Firestore ainsi qu'un système d'authentification, nous avons rencontré des obstacles pour nous connecter à l'émulateur et pour manipuler cet émulateur efficacement. Nous avons d'abord cherché des solutions dans la documentation officielle de Firebase. Malheureusement, nous avons constaté que la documentation concernant l'utilisation de l'émulateur Firebase pour les tests unitaires Firestore était limitée et ne fournissait pas suffisamment d'informations pour résoudre nos problèmes spécifiques. Pour comprendre où se situaient les problèmes, nous avons ajouté des instructions *console.log* dans notre code pour suivre le flux d'exécution et vérifier les valeurs des variables. Nous avons aussi essayé de *mock* la fonction *getDocs* pour simuler son comportement dans l'environnement de test. En dépit de nos tentatives, aucune approche n'a résolu nos problèmes, et les tests unitaires continuent de rencontrer des échecs.

Néanmoins, nous avons laissé nos tests non fonctionnels dans le dossier "src/tests/non fonctionnels".

En conséquence, outre l'impossibilité de tester les méthodes backend, aucun composant ayant un lien avec Firebase, tels que ceux utilisant l'authentification ou les méthodes d'ajout, de modification et de récupération de données ne peut être testé avec succès. Ainsi, l'application et ses fonctionnalités n'ont pas pu être testés de façon exhaustive.

Description des composants et fonctions testés

BoutonOnglets

Le composant *BoutonOnglets* est chargé d'afficher des boutons d'onglets pour la navigation dans une application web. Il prend en entrée une liste d'objets d'onglets contenant des labels et des liens correspondant à leur destination. Ce composant facilite la navigation entre différentes pages ou fonctionnalités de l'application en affichant une barre d'onglets.

Theme

Ce composant permet de basculer entre le mode sombre et le mode clair de l'application en activant ou désactivant un bouton. Il affiche également des icônes représentant le thème actuellement sélectionné.

generateRandomPassword (fonction)

La fonction *generateRandomPassword* est responsable de la génération d'un mot de passe aléatoire d'une longueur spécifiée. Elle est utilisée dans les composants pour générer des mots de passe aléatoires lors de la création de nouveaux comptes utilisateur.

ConcoursListe

ConcoursListe affiche une liste de concours dans une application. Il prend en entrée une liste d'objets de concours, chaque objet représentant un concours avec des détails tels que le nom, le lieu et la date. Ce composant offre des fonctionnalités telles que le tri par date, la sélection d'un concours, et l'affichage des détails de chaque concours sous forme de cartes.

GymnasteListe

Ce composant est responsable de l'affichage d'une liste de gymnastes avec leurs détails. Il prend en charge des fonctionnalités telles que l'affichage des détails des gymnastes, le filtrage par recherche, le tri par nom ou par catégorie, la sélection d'un gymnaste et la gestion de l'état de la carte lorsqu'elle est cliquée.

SocieteListe

SocieteListe affiche une liste de sociétés avec leurs détails comme le nom et l'e-mail. Il permet la fonctionnalité de recherche, de tri et de sélection des sociétés.

UtilisateurListe

Ce composant affiche une liste d'utilisateurs avec leurs détails tels que le nom, le prénom et l'adresse e-mail. Il prend en charge la fonctionnalité de recherche, de tri et de sélection des utilisateurs.

ResultatListe

ResultatListe affiche les résultats d'une compétition pour chaque engin, y compris le classement individuel, les récompenses, le résultat total et les résultats d'équipe le cas échéant. Il prend en compte des fonctionnalités telles que l'affichage des résultats pour chaque engin, le tri par nom d'engin dans l'ordre alphabétique, le calcul du résultat total, l'affichage du classement et des récompenses, ainsi que l'affichage des résultats d'équipe si fournis.

Détails des tests

BoutonOnglets

Test pour vérifier si le composant s'affiche correctement avec les onglets donnés

But du test : Vérifier que le composant affiche correctement les noms des onglets avec les labels fournis.

Préconditions : Créer des objets d'onglets avec un label et un lien.

Étapes du test :

- Rendre le composant BoutonOnglets avec les onglets spécifiés.
- Vérifier la présence des boutons d'onglets dans le rendu.

Résultats attendus : Les boutons d'onglets pour chaque label donné doivent être présents dans le rendu.

Test pour vérifier si le bouton correspondant à l'emplacement actuel est activé

But du test : S'assurer que le bouton correspondant à l'emplacement actuel est activé visuellement.

Préconditions : Créer des objets d'onglets avec un label et un lien et définir l'emplacement initial dans le MemoryRouter.

Étapes du test :

- Rendre le composant BoutonOnglets avec les onglets spécifiés dans un MemoryRouter.
- Vérifier si le bouton correspondant à l'emplacement actuel est activé.

Résultats attendus : Le bouton pour l'onglet dont le lien correspond à l'emplacement actuel doit avoir une classe "active" pour indiquer qu'il est sélectionné.

Test pour vérifier si le clic sur un bouton de l'onglet navigue vers la route correcte

But du test : S'assurer que le clic sur un bouton d'onglet entraîne une navigation vers la route correspondante.

Préconditions : Créer des objets d'onglets avec un label et un lien.

Étapes du test :

- Rendre le composant BoutonOnglets avec les onglets spécifiés.
- Simuler un clic sur un bouton d'onglet.
- Vérifier si la fenêtre est dirigée vers la route correcte après le clic.

Résultats attendus : Après le clic sur un bouton d'onglet, la fenêtre doit être redirigée vers la route correspondante définie pour cet onglet.

Theme

Test pour vérifier l'affichage initial du composant

But du test : S'assurer que le composant est correctement rendu et que les icônes représentant les thèmes sont affichées.

Préconditions : Aucune.

Étapes du test :

- Rendre le composant Theme.
- Vérifier la présence des icônes du thème (soleil et lune).

Résultats attendus : Les icônes du thème (soleil et lune) doivent être présentes dans le composant.

Test pour vérifier le basculement vers le mode sombre

But du test : Vérifier que le thème de l'application bascule vers le mode sombre lorsqu'on active le bouton correspondant.

Préconditions : Aucune.

Étapes du test :

- Rendre le composant Theme.
- Cliquer sur le bouton de basculement vers le mode sombre.

Résultats attendus : Le thème de l'application doit être défini comme sombre.

Test pour vérifier le basculement vers le mode clair

But du test : Vérifier que le thème de l'application bascule vers le mode clair lorsqu'on désactive le bouton correspondant.

Préconditions : Aucune.

Étapes du test :

- Rendre le composant Theme.
- Cliquer sur le bouton de basculement vers le mode sombre pour activer le mode sombre.
- Cliquer sur le bouton de basculement vers le mode clair.

Résultats attendus : Le thème de l'application doit être défini comme clair.

generateRandomPassword

Test de la génération d'un mot de passe aléatoire de la longueur spécifiée

But du test : Vérifier que la fonction generateRandomPassword génère un mot de passe aléatoire de la longueur spécifiée.

Préconditions : Définition d'une longueur spécifique pour le mot de passe.

Étapes du test :

- Appeler la fonction generateRandomPassword avec une longueur spécifiée.
- Récupérer le mot de passe généré.
- Vérifier que la longueur du mot de passe généré correspond à la longueur spécifiée.

Résultats attendus : Le mot de passe généré doit avoir une longueur égale à la longueur spécifiée dans le test.

ConcoursListe

Test d'affichage correct de la liste des concours

But du test : Vérifier que le composant affiche correctement la liste des concours avec leurs détails.

Préconditions : Créer des objets Timestamp pour simuler des dates en utilisant la bibliothèque Firestore et créer des objets de concours avec leurs détails (id, nom, lieu, date).

Étapes du test :

- Rendre le composant ConcoursListe avec les données de test.
- Vérifier la présence des détails de chaque concours dans le rendu.

Résultats attendus : Les détails de chaque concours (nom, lieu et date) doivent être affichés dans le rendu.

Tests du tri des concours par date (ascendante et descendante)

But du test : S'assurer que les concours sont correctement triés en fonction de l'option de tri sélectionnée.

Préconditions : Créer des objets Timestamp pour simuler des dates en utilisant la bibliothèque Firestore et créer des objets de concours avec leurs détails.

Étapes du test :

- Rendre le composant ConcoursListe avec les données de test et une option de tri.
- Récupérer les cartes dans l'ordre du rendu.

Résultats attendus : Les concours doivent être triés correctement, par date ascendante ou descendante.

Test de la sélection d'un concours

But du test : Vérifier que la fonction spécifiée est déclenchée avec les bonnes informations lorsqu'un concours est sélectionné.

Préconditions : Créer des objets Timestamp pour simuler des dates en utilisant la bibliothèque Firestore et créer des objets de concours avec leurs détails. Définir une fonction simulée pour la sélection d'un concours.

Étapes du test :

- Rendre le composant ConcoursListe avec les données de test et la fonction simulée.
- Simuler un clic sur une carte de concours.

Résultats attendus : La fonction simulée doit être appelée avec les informations du concours correspondant à celui sur lequel l'utilisateur a cliqué.

Test de l'état de la carte cliquée

But du test : Vérifier que la classe "cliquee" est ajoutée à la carte lorsque celle-ci est cliquée.

Préconditions : Créer des objets Timestamp pour simuler des dates en utilisant la bibliothèque Firestore et créer des objets de concours avec leurs détails.

Étapes du test :

- Rendre le composant ConcoursListe avec les données de test.
- Simuler un clic sur une carte de concours.

Résultats attendus : La classe "cliquee" doit être ajoutée à la carte cliquée pour indiquer visuellement qu'elle est sélectionnée.

GymnasteListe

Test d'affichage correct de la liste des gymnastes

But du test : Vérifier que le composant affiche correctement la liste des gymnastes avec leurs détails.

Préconditions : Création d'objets de gymnastes avec leurs détails (id, nom, prénom, numéro FSG et catégorie).

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test.
- Vérifier la présence des détails de chaque gymnaste dans le rendu.

Résultats attendus : Les détails de chaque gymnaste (nom, prénom, numéro FSG et catégorie) doivent être affichés correctement dans le rendu.

Test de l'affichage de la liste des gymnastes lorsque display est false

But du test : Vérifier que le composant ne montre pas la liste des gymnastes lorsque display est false.

Préconditions : Création d'objets de gymnastes avec leurs détails.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test et display à false.
- Vérifier qu'aucun gymnaste n'est affiché.

Résultats attendus : Aucun gymnaste ne doit être visible dans le rendu.

Test du filtrage des gymnastes en fonction de la recherche

But du test : Vérifier que le composant filtre correctement les gymnastes en fonction de la recherche.

Préconditions : Création d'objets de gymnastes avec leurs détails.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test et une recherche spécifique.
- Vérifier que seul le gymnaste correspondant à la recherche est affiché.

Résultats attendus : Seul le gymnaste correspondant à la recherche doit être visible dans le rendu.

Test du tri des gymnastes en fonction de l'option de tri sélectionnée (nom)

But du test : Vérifier que le composant trie correctement les gymnastes par nom.

Préconditions : Création d'objets de gymnastes avec leurs détails.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test et une option de tri spécifique par nom.
- Vérifier que les gymnastes sont triés correctement par nom.

Résultats attendus : Les gymnastes doivent être affichés dans l'ordre alphabétique des noms dans le rendu.

Test du tri des gymnastes en fonction de l'option de tri sélectionnée (catégorie)

But du test : Vérifier que le composant trie correctement les gymnastes par catégorie.

Préconditions : Création d'objets de gymnastes avec leurs détails.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test et une option de tri spécifique par catégorie.
- Vérifier que les gymnastes sont triés correctement par catégorie.

Résultats attendus : Les gymnastes doivent être affichés dans l'ordre de leurs catégories dans le rendu.

Test de la sélection d'un gymnaste

But du test : Vérifier que le composant déclenche correctement la fonction onGymnasteClick avec les bonnes informations lorsqu'un gymnaste est sélectionné.

Préconditions : Création d'objets de gymnastes avec leurs détails et d'une fonction de test simulée.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test et la fonction de test simulée.
- Sélectionner un gymnaste et simuler un clic.
- Vérifier que la fonction de test simulée est appelée avec les bonnes informations.

Résultats attendus : La fonction onGymnasteClick doit être appelée une fois avec les informations du gymnaste sélectionné.

Test de l'ajout de la classe "cliquée" à la carte lorsqu'elle est cliquée

But du test : Vérifier que le composant ajoute correctement la classe "cliquée" à la carte lorsqu'elle est cliquée.

Préconditions : Création d'objets de gymnastes avec leurs détails.

Étapes du test :

- Rendre le composant GymnasteListe avec les données de test.
- Sélectionner une carte et simuler un clic.
- Vérifier que la classe "cliquée" est ajoutée à la carte cliquée.

Résultats attendus : La classe "cliquée" doit être ajoutée à la carte cliquée dans le rendu pour indiquer visuellement qu'elle est sélectionnée.

SocieteListe

Test d'affichage correct de la liste des sociétés

But du test : Vérifier que le composant affiche correctement la liste des sociétés avec leurs détails.

Préconditions : Création d'objets de sociétés avec des id, des noms et des adresses e-mail.

Étapes du test :

- Rendre le composant SocieteListe avec les données de test.
- Vérifier la présence des noms et des adresses e-mail de chaque société dans le rendu.

Résultats attendus : Les noms et les adresses e-mail des sociétés doivent être affichés correctement dans le rendu.

Test du filtre des sociétés en fonction de la recherche

But du test : Vérifier que le composant filtre correctement les sociétés en fonction de la recherche.

Préconditions : Création d'objets de sociétés.

Étapes du test :

- Rendre le composant SocieteListe avec les données de test et une recherche spécifique.
- Vérifier que seules les sociétés correspondant à la recherche sont affichées.

Résultats attendus : Seules les sociétés correspondant à la recherche doivent être affichées dans le rendu.

Test du tri des sociétés en fonction de l'option de tri sélectionnée

But du test : Vérifier que le composant trie correctement les sociétés en fonction de l'option de tri sélectionnée.

Préconditions : Création d'objets de sociétés.

Étapes du test :

- Rendre le composant SocieteListe avec les données de test et une option de tri spécifique.
- Vérifier que les sociétés sont affichées dans l'ordre correct selon l'option de tri.

Résultats attendus : Les sociétés doivent être affichées dans l'ordre spécifié par l'option de tri dans le rendu.

Test de la sélection d'une société

But du test : Vérifier que le composant déclenche correctement la fonction spécifiée lorsqu'une société est sélectionnée.

Préconditions : Création d'objets de sociétés, ainsi qu'une fonction de test simulée.

Étapes du test :

- Rendre le composant SocieteListe avec les données de test et la fonction de test simulée.
- Sélectionner une société et simuler le clic.

Résultats attendus : La fonction de test simulée doit être appelée avec une fois avec les informations de la société sélectionnée.

Test de l'état de la carte cliquée

But du test : Vérifier que le composant ajoute correctement la classe "cliquee" à la carte lorsqu'elle est cliquée.

Préconditions : Création d'objets de sociétés avec des noms et des adresses e-mail.

Étapes du test :

- Rendre le composant SocieteListe avec les données de test.
- Sélectionner une carte et simuler le clic.

Résultats attendus : La classe "cliquee" doit être ajoutée à la carte cliquée et non aux autres cartes dans le rendu pour indiquer visuellement une différence.

UtilisateurListe

Test d'affichage correct de la liste des utilisateurs

But du test : Vérifier que le composant affiche correctement la liste des utilisateurs avec leurs détails.

Préconditions : Création d'objets d'utilisateurs avec des id, noms, prénoms et adresses e-mail.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test.
- Vérifier la présence des noms, prénoms et adresses e-mail de chaque utilisateur dans le rendu.

Résultats attendus : Les noms, prénoms et adresses e-mail des utilisateurs doivent être affichés correctement dans le rendu.

Test du filtre des utilisateurs en fonction de la recherche

But du test : Vérifier que le composant filtre correctement les utilisateurs en fonction de la recherche.

Préconditions : Création d'objets d'utilisateurs.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test et une recherche spécifique.
- Vérifier que seul l'utilisateur correspondant à la recherche est affiché.

Résultats attendus : Seul l'utilisateur correspondant à la recherche doit être affiché dans le rendu.

Test du tri des utilisateurs en fonction de l'option de tri sélectionnée (nom)

But du test : Vérifier que le composant trie correctement les utilisateurs en fonction de l'option de tri sélectionnée.

Préconditions : Création d'objets d'utilisateurs.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test et une option de tri "nom".
- Vérifier que les utilisateurs sont affichés dans l'ordre correct selon l'option de tri.

Résultats attendus : Les utilisateurs doivent être affichés dans l'ordre spécifié par l'option de tri dans le rendu (triés par nom).

Test du tri des utilisateurs en fonction de l'option de tri sélectionnée (e-mail)

But du test : Vérifier que le composant trie correctement les utilisateurs en fonction de l'option de tri sélectionnée.

Préconditions : Création d'objets d'utilisateurs.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test et une option de tri "email".
- Vérifier que les utilisateurs sont affichés dans l'ordre correct selon l'option de tri.

Résultats attendus : Les utilisateurs doivent être affichés dans l'ordre spécifié par l'option de tri dans le rendu (triés par email).

Test de la sélection d'un utilisateur

But du test : Vérifier que le composant déclenche correctement la fonction spécifiée lorsqu'un utilisateur est sélectionné.

Préconditions : Création d'objets d'utilisateurs, ainsi qu'une fonction de test simulée.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test et la fonction de test simulée.
- Sélectionner un utilisateur et simuler le clic.

Résultats attendus : La fonction de test simulée doit être appelée avec les informations de l'utilisateur sélectionné.

Test de l'état de la carte cliquée

But du test : Vérifier que le composant ajoute correctement la classe "cliquee" à la carte lorsqu'elle est cliquée.

Préconditions : Création d'objets d'utilisateurs.

Étapes du test :

- Rendre le composant UtilisateurListe avec les données de test.
- Sélectionner une carte et simuler le clic.

Résultats attendus : La classe "cliquee" doit être ajoutée à la carte cliquée et non aux autres cartes dans le rendu.

ResultatListe

Test d'affichage correct des résultats pour chaque engin

But du test : Vérifier que le composant affiche correctement la liste des résultats pour chaque engin avec leurs valeurs correspondantes.

Préconditions : Création d'objets de résultats pour chaque engin avec des valeurs numériques.

Étapes du test :

- Rendre le composant ResultatListe avec les données de test.
- Vérifier la présence des noms d'engins et de leurs résultats correspondants dans le rendu.

Résultats attendus : Les noms d'engins et leurs résultats doivent être affichés correctement dans le rendu.

Test de tri des résultats par clé d'engin dans l'ordre alphabétique

But du test : Vérifier que le composant trie correctement les résultats par nom d'engin dans l'ordre alphabétique.

Préconditions : Création d'objets de résultats pour chaque engin avec des valeurs numériques.

Étapes du test :

- Rendre le composant ResultatListe avec les données de test.
- Vérifier que les résultats sont triés par clé d'engin dans l'ordre alphabétique.

Résultats attendus : Les résultats doivent être affichés dans l'ordre alphabétique des noms d'engins dans le rendu.

Test de calcul correct du résultat total

But du test : Vérifier que le composant calcule correctement le résultat total à partir des résultats individuels.

Préconditions : Création d'objets de résultats pour chaque engin avec des valeurs numériques.

Étapes du test :

- Rendre le composant ResultatListe avec les données de test.
- Vérifier que le résultat total est correct.

Résultats attendus : Le résultat total doit être calculé correctement et affiché dans le rendu.

Test d'affichage correct du classement et des récompenses

But du test : Vérifier que le composant affiche correctement le classement et les récompenses.

Préconditions : Création d'objets de résultats pour chaque engin avec des valeurs numériques.

Étapes du test :

- Rendre le composant ResultatListe avec les données de test.
- Vérifier que le classement et la récompense sont affichés correctement.

Résultats attendus : Le classement et la récompense doivent être affichés correctement dans le rendu.

Test d'affichage correct des résultats d'équipe si fournis

But du test : Vérifier que le composant affiche correctement les résultats d'équipe si fournis.

Préconditions : Création d'objets de résultats pour chaque engin avec des valeurs numériques.

Étapes du test :

- Rendre le composant ResultatListe avec les données de test et des données d'équipe (total et classement).
- Vérifier que les résultats d'équipe sont affichés correctement.

Résultats attendus : Les résultats d'équipe doivent être affichés correctement dans le rendu.

Réussite des Tests

Lorsque nous lançons les tests depuis le terminal, nous voyons qu'ils s'exécutent tous avec succès :

```
PS C:\Users\... \PSM_DataAccrosBates> npm test

> psm-dataaccrosbates@0.0.0 test
> jest

PASS src/tests/PasswordGenerator.test.js
PASS src/tests/SocieteListe.test.js
PASS src/tests/UtilisateurListe.test.js
PASS src/tests/GymnasteListe.test.js
PASS src/tests/BoutonOnlgets.test.js
PASS src/tests/ResultatListe.test.js
PASS src/tests/Theme.test.js
PASS src/tests/ConcoursListe.test.js

Test Suites: 8 passed, 8 total
Tests:       35 passed, 35 total
Snapshots:   0 total
Time:        5.397 s, estimated 17 s
Ran all test suites.
```

Conclusion

Les tests effectués ont permis de valider le bon fonctionnement des composants et des fonctionnalités de l'application web, en garantissant leur conformité aux spécifications et en assurant une expérience utilisateur fluide. Voici un résumé des principaux résultats des tests :

- Les composants tels que *BoutonOnglets*, *ConcoursListe*, *GymnasteListe*, *SocieteListe*, *UtilisateurListe*, *ResultatListe* et *Theme* ont été testés avec succès, démontrant leur capacité à afficher correctement les données, à gérer les interactions utilisateur et à répondre aux différentes fonctionnalités attendues.
- La fonction *generateRandomPassword* a également été testée avec succès, assurant la génération fiable de mots de passe aléatoires, renforçant ainsi la sécurité de l'application.

Cependant, il est important de noter que tous les aspects de l'application n'ont pas pu être testés dans le cadre de cette série de tests. Des défis techniques, notamment liés à l'intégration avec Firestore, ont limité notre capacité à tester certaines fonctionnalités backend et les composants associés.